

Application mobile de gestion médicale

Documentation technique

Description des choix techniques

1. Inscription/Connexion

Implémentation :

- **Frontend** : Formulaire dans une page de connexion/inscription, appels à /register et /login via AuthService.
- **Backend** : /register hache le mot de passe et insère dans patient. /login vérifie les identifiants et génère un token JWT.

Extrait de code (app.py) :

```
# Route d'inscription (patients)

@app.route('/register', methods=['POST'])

def register():

    data = request.json

    if not data.get("email") or not data.get("password") or not data.get("name"):

        return jsonify({"message": "Nom complet, email et mot de passe requis"}), 400

    if patients.find_one({"email": data["email"]}) or medecins.find_one({"email": data["email"]}):

        return jsonify({"message": "Utilisateur déjà existant"}), 400

    hashed_pw = bcrypt.generate_password_hash(data["password"]).decode('utf-8')

    patient_data = {

        "name": data["name"],

        "email": data["email"],

        "password": hashed_pw,
```

```

        "role": "patient"
    }

    patients.insert_one(patient_data)

    return jsonify({"message": "Inscription réussie"}), 201

```

Extrait de code (auth.service.ts) :

```

// Inscription

register(user: any): Observable<any> {

    return this.http.post<any>(`${this.apiUrl}/register`, user);

}

```

Extrait de code (register.page.ts) :

```

register() {

    if (!this.name || !this.email || !this.password) {

        alert('Veuillez remplir tous les champs.');
```

```

        return;

    }

    const user = {

        name: this.name,

        email: this.email,

        password: this.password,

        role: 'patient' // Toujours un patient

    };

    this.authService.register(user).subscribe(

        (res) => {

            alert(res.message || 'Inscription réussie !');
```

```

            this.router.navigate(['/login']);

```

```

    },

    (err) => {

        alert(err.error?.message || 'Erreur lors de l'inscription.');
```

Fichiers liés :

- **Frontend** : login.page.ts, login.page.html, auth.service.ts.
- **Backend** : app.py, collections patient, medecin.

Choix techniques :

- **JWT** : Sécurisation via tokens.
- **Bcrypt** : Hachage sécurisé des mots de passe.
- **Ionic Alerts** : Affichage des messages d'erreur/succès.
- **Angular HttpClient** : Requêtes HTTP asynchrones.

2. Recherche de médecins

Implémentation :

- **Frontend** : home.page.ts utilise RendezVousService pour appeler /medecins/search avec des paramètres de filtrage (spécialité, adresse, terme). Les résultats sont affichés dans home.page.html.
- **Backend** : /medecins/search filtre les médecins avec regex et inclut les disponibilités valides.

Extrait de code (app.py):

```

# Route pour la recherche des médecins

@app.route('/medecins/search', methods=['GET'])

def search_medecins():
```

```
try:

    specialite = request.args.get('specialite', '').lower().strip()

    gouvernorat = request.args.get('gouvernorat', '').lower().strip()

    ville = request.args.get('ville', '').lower().strip()

    term = request.args.get('term', '').lower().strip()


    query = {}

    if specialite:

        query["specialite"] = {"$regex": specialite, "$options": "i"}

    if gouvernorat:

        query["gouvernorat"] = {"$regex": gouvernorat, "$options":
    "i"}

    if ville:

        query["ville"] = {"$regex": ville, "$options": "i"}

    if term:

        query["$or"] = [

            {"name": {"$regex": term, "$options": "i"}},

            {"email": {"$regex": term, "$options": "i"}},

            {"specialite": {"$regex": term, "$options": "i"}},

            {"gouvernorat": {"$regex": term, "$options": "i"}},

            {"ville": {"$regex": term, "$options": "i"}},

            {"adresse": {"$regex": term, "$options": "i"}},

            {"tel": {"$regex": term, "$options": "i"}},
```

```
        {"gsm": {"$regex": term, "$options": "i"}},
        {"sexe": {"$regex": term, "$options": "i"}},
        {"etab": {"$regex": term, "$options": "i"}},
        {"faculte": {"$regex": term, "$options": "i"}}
    ]

    medecins_list = list(medecins.find(query, {"password": 0}))
```

extrait de home.page.ts:

```
loadFilterOptions() {
    this.rdvService.getSpecialites().subscribe({
        next: (data) => this.specialites = data,
        error: (err) => console.error('Erreur lors du chargement des spécialités:', err)
    });

    this.rdvService.getGouvernorats().subscribe({
        next: (data) => this.gouvernorats = data,
        error: (err) => console.error('Erreur lors du chargement des gouvernorats:', err)
    });

    this.rdvService.getVilles().subscribe({
```

```
    next: (data) => this.villes = data,

    error: (err) => console.error('Erreur lors du chargement des
villes:', err)

  });

}

toggleFilters() {

  this.showFilters = !this.showFilters;

}

filterMedecins() {

  const params = {

    specialite: this.filters.specialite.trim(),

    gouvernorat: this.filters.gouvernorat.trim(),

    ville: this.filters.ville.trim(),

    term: this.filters.term.trim()

  };

  if (!params.specialite && !params.gouvernorat && !params.ville &&
!params.term) {

    this.medecins = [...this.allMedecins];

    return;

  }
}
```

```

this.rdvService.searchMedecins(params).subscribe({
  next: (data: any[]) => {
    this.medecins = data.map(medecin => ({
      ...medecin,
      showDisponibilites: false,
      disponibilites: medecin.disponibilites || {}
    }));
  },
  error: (err) => {
    console.error('Erreur lors du filtrage des médecins:', err);
    this.showAlert('Erreur', 'Impossible de filtrer les médecins.');
  }
});

resetFilters() {
  this.filters = { specialite: '', gouvernorat: '', ville: '', term: '' };
  this.medecins = [...this.allMedecins];
}

```

extrait de rendez-vous.service.ts:

```

// Nouvelle méthode pour la recherche filtrée

searchMedecins(params: { specialite?: string, adresse?: string, term?:
string }): Observable<any[]> {

```

```

        return this.http.get<any[]>(`${this.apiUrl}/medecins/search`, {
params });
    }

    getGouvernorats(): Observable<string[]> {
        return this.http.get<string[]>(`${this.apiUrl}/gouvernorats`);
    }

    getVilles(): Observable<string[]> {
        return this.http.get<string[]>(`${this.apiUrl}/villes`);
    }

    getSpecialites(): Observable<string[]> {
        return this.http.get<string[]>(`${this.apiUrl}/specialites`);
    }
}

```

Fichiers liés :

- **Frontend** : home.page.ts, home.page.html, rendez-vous.service.ts.
- **Backend** : app.py, collections medecin, disponibilite.

Choix techniques :

- **MongoDB Regex** : Recherche insensible à la casse.
- **Angular Observables** : Gestion asynchrone des résultats.
- **Ionic UI** : Filtres interactifs avec boutons.

3. Prise de rendez-vous

Implémentation :

- **Frontend** : Dans `home.page.ts`, `prendreRendezVous` utilise des alertes Ionic pour sélectionner un jour et un créneau, validés par regex. `RendezVousService` envoie la requête à `/rendezvous`.
- **Backend** : `/rendezvous` vérifie la disponibilité, marque le créneau comme indisponible, et enregistre le rendez-vous.

extrait de `app.py`:

```
if request.method == 'POST':

    data = request.get_json()

    if claims["role"] != "patient" or not all(k in data for k in
["medecinId", "jour", "debut", "fin"]):

        return jsonify({"message": "Données invalides ou accès
refusé"}), 400

    medecin_id = data["medecinId"]

    jour = data["jour"].lower()

    debut = data["debut"]

    fin = data["fin"]

    dispo = disponibilites.find_one({"medecinId": medecin_id})

    if not dispo or jour not in dispo["horaires"]:
```

```

        return jsonify({"message": "Créneau non disponible"}), 400

    creneaux = dispo["horaires"][jour]

    creneau_index = next(
        (i for i, c in enumerate(creneaux)
         if c["debut"] == debut and c["fin"] == fin and
c["est_disponible"])),
        None
    )

    if creneau_index is None:
        return jsonify({"message": "Créneau non disponible ou déjà
réservé"}), 409

    dispo["horaires"][jour][creneau_index]["est_disponible"] = False

    disponibilites.update_one(
        {"medecinId": medecin_id},
        {"$set": {f"horaires.{jour}": dispo["horaires"][jour]}}
    )

    rdv = {
        "medecinId": data["medecinId"],
        "patientId": user_id,
        "jour": data["jour"],
        "debut": data["debut"],

```

```

        "fin": data["fin"],

        "statut": "confirmé",

        "date_creation": datetime.now().isoformat()

    }

    result = rendezvous.insert_one(rdv)

    medecin = medecins.find_one({"_id": ObjectId(data["medecinId"])})

    patient = patients.find_one({"_id": ObjectId(user_id)})

    if medecin and patient:

        create_notification(data["medecinId"],

                            f"{patient['name']} a pris un rendez-vous le {data['jour']} de {data['debut']} à {data['fin']}.",

                            "rendezvous")

        return jsonify({"message": "Rendez-vous pris", "id":
str(result.inserted_id)}), 201

```

extrait de home.page.ts:

```

async prendreRendezVous(medecinId: string) {

    if (!this.isLoggedIn || this.userRole !== 'patient') {

        const alert = await this.alertCtrl.create({

            header: 'Erreur',

            message: 'Vous devez être connecté en tant que patient pour
prendre un rendez-vous.',

            buttons: ['OK']

        });
    }
}

```

```

    await alert.present();

    return;
}

const medecin = this.medecins.find(m => m.id === medecinId);

if (!medecin) {

    const alert = await this.alertCtrl.create({

        header: 'Erreur',

        message: 'Médecin non trouvé.',

        buttons: ['OK']

    });

    await alert.present();

    return;

}

const joursDisponibles = Object.keys(medecin.disponibilites).filter(

    jour => this.validDays.includes(jour) &&
medecin.disponibilites[jour]?.length > 0

);

if (joursDisponibles.length === 0) {

    const alert = await this.alertCtrl.create({

        header: 'Erreur',

```

```

        message: 'Aucune disponibilité pour ce médecin.',
        buttons: ['OK']
    });

    await alert.present();

    return;
}

const jourAlert = await this.alertCtrl.create({
    header: 'Choisir un jour',
    inputs: joursDisponibles.map(jour => ({
        name: 'jour',
        type: 'radio',
        label: jour.charAt(0).toUpperCase() + jour.slice(1),
        value: jour
    })),
    buttons: [
        { text: 'Annuler', role: 'cancel' },
        {
            text: 'Suivant',
            handler: async (selectedJour: string) => {
                if (!selectedJour) {
                    const errorAlert = await this.alertCtrl.create({
                        header: 'Erreur',

```

```

        message: 'Veuillez sélectionner un jour.',

        buttons: ['OK']

    });

    await errorAlert.present();

    return false;
}

const creneaux = medecin.disponibilites[selectedJour] || [];

const creneauxValides = creneaux.filter(

    (creneau: any) => creneau.debut && creneau.fin &&
/^\\d{1,2}:\\d{2}$/.test(creneau.debut) &&
/^\\d{1,2}:\\d{2}$/.test(creneau.fin)

);

if (creneauxValides.length === 0) {

    const errorAlert = await this.alertCtrl.create({

        header: 'Erreur',

        message: `Aucun créneau valide disponible pour
${selectedJour}.`,

        buttons: ['OK']

    });

    await errorAlert.present();

    return false;

}

```

```

const creneauAlert = await this.alertCtrl.create({

  header: `Créneaux pour
${selectedJour.charAt(0).toUpperCase() + selectedJour.slice(1)}`,

  inputs: creneauxValides.map((creneau: any) => ({

    name: 'creneau',

    type: 'radio',

    label: `${creneau.debut} - ${creneau.fin}`,

    value: `${creneau.debut}|${creneau.fin}`

  })),

  buttons: [

    { text: 'Annuler', role: 'cancel' },

    {

      text: 'Confirmer',

      handler: async (selectedCreneau: string) => {

        if (!selectedCreneau) {

          const errorAlert = await this.alertCtrl.create({

            header: 'Erreur',

            message: 'Veuillez sélectionner un créneau.',

            buttons: ['OK']

          });

          await errorAlert.present();

          return false;

```

```
}

const [debut, fin] = selectedCreneau.split('|');

const rdv = {

  medecinId: medecinId,

  jour: selectedJour.toLowerCase(),

  debut,

  fin

};

this.rdvService.prendreRendezVous(rdv).subscribe({

  next: async () => {

    const successAlert = await this.alertCtrl.create({

      header: 'Succès',

      message: 'Rendez-vous pris avec succès !',

      buttons: ['OK']

    });

    await successAlert.present();

    this.loadMedecins();

  },

  error: async (err: any) => {

    console.error('Erreur lors de la prise de rendez-vous:', err);
```



```

        const errorAlert = await this.alertCtrl.create({
            header: 'Erreur',
            message: err.error?.message || 'Erreur lors de
la prise de rendez-vous.',
            buttons: ['OK']
        });

        await errorAlert.present();
    }

    });

    return true;
}

}

]

});

    await creneauAlert.present();

    return true;
}

}

]

});

    await jourAlert.present();
}

```

extrait de rendez-vous.service.ts:

```

getRendezVous(): Observable<any[]> {

```

```

return this.http.get<any[]>(`${this.apiUrl}/rendezvous`, { headers:
this.getHeaders() });

    }

    prendreRendezVous(rdv: any): Observable<any> {

        return this.http.post(`${this.apiUrl}/rendezvous`, rdv, { headers:
this.getHeaders() });

    }

    annulerRendezVous(rdvId: string): Observable<any> {

        return
this.http.post(`${this.apiUrl}/rendezvous/${rdvId}/annuler`, {}, {
headers: this.getHeaders() });

    }

```

Fichiers liés :

- **Frontend** : home.page.ts, home.page.html, rendez-vous.service.ts.
- **Backend** : app.py, collections disponibilite, rendezvous.

Choix techniques :

- **Ionic Alerts** : Sélection intuitive des créneaux.
- **Regex** : Validation des formats horaires.
- **JWT** : Restriction aux patients.

4. Gestion des disponibilités

Implémentation :

- **Frontend** : Une page disponibilites.page.ts (supposée) permet aux médecins d'ajouter des créneaux, envoyés via RendezVousService à /disponibilites.
- **Backend** : /disponibilites valide les créneaux et met à jour la collection disponibilite.

extrait de app.py:

```
# Gestion des disponibilités

@app.route('/disponibilites', methods=['GET', 'POST'])
@jwt_required()
def manage_disponibilites():

    user_id = get_jwt_identity()

    claims = get_jwt()

    print(f"User_id: {user_id}, Role: {claims['role']}")

    if claims["role"] != "medecin":

        return jsonify({"message": "Accès réservé aux médecins"}), 403

    if request.method == 'GET':

        doc = disponibilites.find_one({"medecinId": user_id})

        default = {"lundi": [], "mardi": [], "mercredi": [], "jeudi": [],
"vendredi": [], "samedi": []}

        return jsonify(doc["horaires"] if doc else default), 200

    if request.method == 'POST':

        data = request.get_json()

        if not isinstance(data, dict):

            return jsonify({"message": "Données invalides"}), 400
```

```

        valid_days = ["lundi", "mardi", "mercredi", "jeudi", "vendredi",
"samedi"]

        updated_horaires = {}

        for jour, creneaux in data.items():

            if jour not in valid_days:

                continue

            updated_creneaux = []

            for creneau in creneaux:

                if "debut" in creneau and "fin" in creneau:

                    updated_creneaux.append({

                        "debut": creneau["debut"],

                        "fin": creneau["fin"],

                        "est_disponible": True

                    })

            updated_horaires[jour] = updated_creneaux

        if not updated_horaires:

            return jsonify({"message": "Aucun créneau valide fourni"}),

400

    disponibilites.update_one(

        {"medecinId": user_id},

        {"$set": {"horaires": updated_horaires}},

```

```

        upsert=True
    )

    return jsonify({"message": "Disponibilités mises à jour"}), 200

```

extrait de espace-medecin.ts:

```

async ajouterCreneau(jour: string) {

    const alert = await this.alertCtrl.create({

        header: `Ajouter un créneau pour ${jour}`,

        inputs: [

            { name: 'debut', type: 'time', placeholder: '08:00' },

            { name: 'fin', type: 'time', placeholder: '12:00' }

        ],

        buttons: [

            { text: 'Annuler', role: 'cancel' },

            {

                text: 'Ajouter',

                handler: (data) => {

                    if (data.debut && data.fin) {

                        this.disponibilites[jour].push({ debut:
data.debut, fin: data.fin });

                        return true;

                    }

                    return false;

                }

            }

        ]

    });
}

```

```

        }

    ]

    });

    await alert.present();

}

```

extrait de rendez-vous.service.ts:

```

getDisponibilites(): Observable<any> {

    return this.http.get(`${this.apiUrl}/disponibilites`, { headers:
this.getHeaders() });

}

saveDisponibilites(horaires: any): Observable<any> {

    return this.http.post(`${this.apiUrl}/disponibilites`, horaires, {
headers: this.getHeaders() });

}

```

5. Gestion des documents

Implémentation :

- **Frontend** : Une page documents.page.ts permet aux patients d'uploader des fichiers et aux médecins de les consulter/télécharger via DocumentsService.
- **Backend** : Les routes /documents/upload et /documents/medecin gèrent le stockage dans SQLite.

dans app.py:

```

# Fonction pour obtenir une connexion SQLite

def get_sqlite_connection():

```

```

conn = sqlite3.connect('documents.db')

conn.row_factory = sqlite3.Row

return conn

# Création de la table au démarrage
def init_sqlite_db():

    with get_sqlite_connection() as conn:

        conn.execute('''

            CREATE TABLE IF NOT EXISTS documents (

                id INTEGER PRIMARY KEY AUTOINCREMENT,

                patient_id TEXT NOT NULL,

                medecin_id TEXT NOT NULL,

                titre TEXT NOT NULL,

                fichier BLOB NOT NULL,

                date_envoi DATETIME DEFAULT CURRENT_TIMESTAMP,

                statut TEXT DEFAULT 'Non consulté',

                remarques TEXT

            )

        ''')

        conn.commit()

        print("Table 'documents' créée ou déjà existante.")

# Routes pour les documents (SQLite)

@app.route('/documents/upload', methods=['POST'])

```

```

@jwt_required()

def upload_document():

@app.route('/documents/patient', methods=['GET'])

@jwt_required()

def get_patient_documents():

@app.route('/documents/medecin', methods=['GET'])

@jwt_required()

def get_medecin_documents():

@app.route('/documents/<int:doc_id>/download', methods=['GET'])

@jwt_required()

def download_document(doc_id):

@app.route('/documents/<int:doc_id>/annotate', methods=['PUT'])

@jwt_required()

def annotate_document(doc_id):

```

dans documents.service.ts:

```

uploadDocument(formData: FormData): Observable<any> {

    return this.http.post(`${this.apiUrl}/documents/upload`, formData, {
headers: this.getHeaders() });

}

getPatientDocuments(): Observable<any[]> {

    return this.http.get<any[]>(`${this.apiUrl}/documents/patient`, {
headers: this.getHeaders() });

}

```



```

getMedecinDocuments(): Observable<any[]> {
    return this.http.get<any[]>(`${this.apiUrl}/documents/medecin`, {
headers: this.getHeaders() });
}

downloadDocument(docId: number): Observable<Blob> {
    return this.http.get(`${this.apiUrl}/documents/${docId}/download`, {
        headers: this.getHeaders(),
        responseType: 'blob'
    });
}

annotateDocument(docId: number, remarques: string): Observable<any> {
    return this.http.put(`${this.apiUrl}/documents/${docId}/annotate`, {
remarques }, { headers: this.getHeaders() });
}

```

dans documents.page.ts:

```

@Component({
    selector: 'app-documents',
    templateUrl: './documents.page.html',
    styleUrls: ['./documents.page.scss'],
})
export class DocumentsPage implements OnInit {

```

```

// === Attributs importants ===

user: any;                // Utilisateur connecté (rôle, id...)

documents: any[] = [];    // Liste des documents affichés

selectedFile: File | null; // Fichier sélectionné à uploader

titre: string;            // Titre du document

selectedMedecinId: string; // ID du médecin concerné

medecins: any[] = [];     // Liste des médecins associés au patient


constructor(

    private authService: AuthService,          // Gestion de
l'authentification

    private documentsService: DocumentsService, // Appels backend pour
les documents

    private alertCtrl: AlertController         // Affichage des alertes
Ionic

) {}


// === Méthodes principales ===


ngOnInit() {

    // Initialiser les données utilisateur et charger documents/médecins

}

```

```
loadDocuments() {  
    // Charger les documents selon le rôle (patient/médecin)  
}  
  
loadMedecins() {  
    // Récupérer la liste des médecins liés à un patient  
}  
  
onFileSelected(event: any) {  
    // Enregistrer le fichier choisi dans selectedFile  
}  
  
uploadDocument() {  
    // Vérifier les champs et envoyer un document au backend  
}  
  
downloadDocument(docId: number) {  
    // Télécharger un document via son ID  
}  
  
annotateDocument(doc: any) {  
    // Ajouter une remarque à un document existant
```

```
}  
  
}
```

Fichiers liés :

- **Frontend** : documents.page.ts, documents.page.html, documents.service.ts.
- **Backend** : app.py, documents.db.

Choix techniques :

- **SQLite** : Stockage des fichiers binaires.
- **FormData** : Gestion des uploads de fichiers.
- **Ionic Alerts** : Feedback utilisateur.

6. Notifications

Implémentation :

- **Frontend** : Une page notifications.page.ts affiche les notifications via DocumentsService et permet de les marquer comme lues ou les supprimer. Aussi une icône de notifications permet d'afficher, marquer comme lues et supprimer utilisée dans le header.component.
- **Backend** : /notifications récupère les notifications, /notifications/<id>/read met à jour leur statut.

dans app.py:

```
# Route pour les notifications  
  
@app.route('/notifications', methods=['GET'])  
  
@jwt_required()  
  
def get_notifications():  
  
    user_id = get_jwt_identity()  
  
    notifs = list(notifications.find({"user_id":  
user_id}).sort("created_at", -1))
```

```

        result = [{"id": str(n["_id"]), "message": n["message"], "type":
n["type"], "created_at": n["created_at"].isoformat(), "read": n["read"]}
for n in notifs]

        return jsonify(result), 200

@app.route('/notifications/<notif_id>/read', methods=['PUT'])

@jwt_required()

def mark_notification_read(notif_id):

    user_id = get_jwt_identity()

    result = notifications.update_one(

        {"_id": ObjectId(notif_id), "user_id": user_id},

        {"$set": {"read": True}}

    )

    if result.modified_count == 0:

        return jsonify({"message": "Notification non trouvée ou déjà
lue"}), 404

    return jsonify({"message": "Notification marquée comme lue"}), 200

```

dans documents.service.ts:

```

// Méthodes pour les notifications (inchangées)

getNotifications(): Observable<any[]> {

    return this.http.get<any[]>(`${this.apiUrl}/notifications`, { headers:
this.getHeaders() });
}

```

```

}

markNotificationRead(notifId: string): Observable<any> {

    return this.http.put(`${this.apiUrl}/notifications/${notifId}/read`,
    {}, { headers: this.getHeaders() });

}

getUnreadNotificationsCount(): Observable<number> {

    return this.getNotifications().pipe(

        map(notifications => notifications.filter(n => !n.read).length)

    );

}

```

dans notifications-page.page.ts:

```

loadNotifications() {

    this.documentsService.getNotifications().subscribe({

        next: (data) => {

            this.notifications = data;

        },

        error: (err) => console.error(err)

    });

}

```

```

markAsRead(notifId: string) {

    this.documentsService.markNotificationRead(notifId).subscribe(() => {

        this.loadNotifications();

    });

}

removeNotification(notifId: string) {

    const hidden = JSON.parse(localStorage.getItem('hiddenNotifs') || '[]');

    if (!hidden.includes(notifId)) {

        hidden.push(notifId);

        localStorage.setItem('hiddenNotifs', JSON.stringify(hidden));

    }

}

```

dans notifications.component.ts:

```

loadNotifications() {

    const hidden = JSON.parse(localStorage.getItem('hiddenNotifs') || '[]');

    this.documentsService.getNotifications().subscribe({

```

```

    next: (data) => {

        this.notifications = data.filter((notif: any) =>
!hidden.includes(notif.id));

    },

    error: (err) => console.error(err)

});

}

markAsRead(notifId: string) {

    this.documentsService.markNotificationRead(notifId).subscribe({

        next: () => {

            this.loadNotifications();

        },

        error: (err) => console.error(err)

    });

}

async closePopover() {

    // Retirer le focus avant de fermer

    const activeElement = document.activeElement as HTMLElement;

    if (activeElement) {

        activeElement.blur();
    }
}

```



```

    }

    await this.popoverController.dismiss();
}

removeNotification(notifId: string) {

    const hidden = JSON.parse(localStorage.getItem('hiddenNotifs') || '[]');

    if (!hidden.includes(notifId)) {

        hidden.push(notifId);

        localStorage.setItem('hiddenNotifs', JSON.stringify(hidden));

    }

    this.notifications = this.notifications.filter(notif => notif.id !== notifId);

}

async navigateTo(route: string) {

    // Retirer le focus avant de naviguer

    const activeElement = document.activeElement as HTMLElement;

    if (activeElement) {

        activeElement.blur();

    }

    await this.router.navigate([route]);
}

```

```
await this.popoverController.dismiss();  
  
}
```

Fichiers liés :

- **Frontend** : notifications.page.ts, notifications.page.html, notifications.component.ts, header.component.ts, documents.service.ts.
- **Backend** : app.py, collection notifications.

Choix techniques :

- **MongoDB** : Stockage avec tri par date.
- **ionic** : Interface simple pour afficher/marker les notifications.
- **JWT** : Restriction aux notifications de l'utilisateur.

7. Gestion du profil

Implémentation :

- **Frontend** : Une page parametres.page.ts permet de consulter, mettre à jour le profil et supprimer le compte via AuthService.
- **Backend** : /user/profile (GET/PUT) gère la récupération et la mise à jour des données et /user/delete supprime l'utilisateur et ses rendez-vous.

dans app.py:

```
# === ROUTES POUR LA GESTION DU PROFIL UTILISATEUR (patient / médecin) ===  
  
# Récupérer le profil utilisateur connecté  
  
@app.route('/user/profile', methods=['GET'])  
  
@jwt_required()
```

```
def get_user_profile():

    user_id = get_jwt_identity()    # Récupérer l'ID de l'utilisateur

    claims = get_jwt()              # Récupérer les claims (rôle)

    # Selon le rôle, chercher le profil dans la bonne collection MongoDB

    if role == "medecin":

        user = chercher_dans_collection(medecins, user_id)

    elif role == "patient":

        user = chercher_dans_collection(patients, user_id)

    else:

        return erreur("Rôle non reconnu")

    return user or erreur("Utilisateur non trouvé")

# Mettre à jour les informations du profil

@app.route('/user/profile', methods=['PUT'])

@jwt_required()

def update_user_profile():

    user_id = get_jwt_identity()

    claims = get_jwt()

    data = request.get_json()
```

```
# Vérification des champs requis (email + password)

if not champs_valides(data, ["email", "password"]):

    return erreur("Email et mot de passe requis")


# Vérifier l'identité de l'utilisateur

collection = medecins if claims["role"] == "medecin" else patients
user = verifier_utilisateur(collection, user_id, data)

if not user:

    return erreur("Authentification échouée")


# Filtrer les champs autorisés à mettre à jour

champs_autorises = {

    "medecin": [...], # nom, email, spécialité, etc.

    "patient": ["name", "email"]

}

updates = filtrer_champs(data, champs_autorises[claims["role"]])

if not updates:

    return erreur("Aucun champ valide fourni")
```

```
# Effectuer la mise à jour

result = collection.update_one({"_id": ObjectId(user_id)}, {"$set":
updates})

if result.matched_count == 0:

    return erreur("Utilisateur non trouvé")

return succès("Profil mis à jour avec succès")

# Supprimer le compte utilisateur

@app.route('/user/delete', methods=['DELETE'])

@jwt_required()

def delete_user():

    user_id = get_jwt_identity()

    claims = get_jwt()

    data = request.get_json()

    # Vérification des champs requis

    if not champs_valides(data, ["email", "password"]):

        return erreur("Email et mot de passe requis")

    # Vérifier utilisateur
```

```

collection = medecins if claims["role"] == "medecin" else patients

user = verifier_utilisateur(collection, user_id, data)

if not user:

    return erreur("Authentification échouée")

# Supprimer l'utilisateur et ses rendez-vous liés
collection.delete_one({"_id": ObjectId(user_id)})
rendezvous.delete_many({"_id": ObjectId(user_id)})

return succès("Compte supprimé avec succès")

```

dans `parametres.page.ts`:

```

loadUserProfile() {

    console.log('Chargement du profil...');

    this.http.get('http://127.0.0.1:5000/user/profile', {

        headers: { 'Authorization': `Bearer ${this.authService.getToken()}`

    })

    }).subscribe({

        next: (data: any) => {

            console.log('Données reçues du backend:', data);

            this.userData = { ...data };

```

```

        this.user = { ...data };
    },
    error: (err) => {
        console.error('Erreur lors du chargement du profil:', err);
        this.showAlert('Erreur', 'Impossible de charger vos données.');
```

}

```

    });
}

toggleEdit() {
    this.isEditing = !this.isEditing;

    if (!this.isEditing) {
        this.userData = { ...this.user };
    }

    console.log('Mode édition:', this.isEditing, 'Données actuelles:',
this.userData);
}

async saveChanges() {
    const alert = await this.alertCtrl.create({
        header: 'Confirmer la modification',
        message: 'Veuillez entrer votre email et mot de passe pour
confirmer.',
        inputs: [
```

```
{
  name: 'email',
  type: 'email',
  placeholder: 'Votre email'
},
{
  name: 'password',
  type: 'password',
  placeholder: 'Votre mot de passe'
}
],
buttons: [
  { text: 'Annuler', role: 'cancel' },
  {
    text: 'Confirmer',
    handler: (data) => {
      if (!data.email || !data.password) {
        this.showAlert('Erreur', 'Email et mot de passe requis.');


return false;


      }
    }
  }
]
// Ajouter email et mot de passe aux données envoyées
const payload = {
```



```

        ...this.userData,

        email: data.email,

        password: data.password

    };

    this.http.put('http://127.0.0.1:5000/user/profile', payload, {

        headers: { 'Authorization': `Bearer
${this.authService.getToken()}` }

    }).subscribe({

        next: () => {

            this.user = { ...this.userData };

            this.authService.logout();

            this.showAlert('Succès', 'Profil mis à jour avec succès.
Veuillez vous reconnecter.', () => {

                this.router.navigate(['/login']);

            });

        },

        error: (err) => {

            console.error('Erreur lors de la mise à jour:', err);

            const message = err.error?.message || 'Erreur lors de la
mise à jour. Vérifiez vos identifiants.';

            this.showAlert('Erreur', message);

        }

    });

```

```

        return true;
    }
}

]);

await alert.present();

this.isEditing = false;
}

async deleteAccount() {
    const confirmAlert = await this.alertCtrl.create({
        header: 'Supprimer le compte',
        message: 'Êtes-vous sûr de vouloir supprimer votre compte ? Cette action est irréversible.',
        buttons: [
            { text: 'Annuler', role: 'cancel' },
            {
                text: 'Oui, supprimer',
                handler: async () => {
                    const credAlert = await this.alertCtrl.create({
                        header: 'Confirmer la suppression',
                        message: 'Veuillez entrer votre email et mot de passe pour confirmer.',
                        inputs: [

```

```

        {
            name: 'email',
            type: 'email',
            placeholder: 'Votre email'
        },
        {
            name: 'password',
            type: 'password',
            placeholder: 'Votre mot de passe'
        }
    ],
    buttons: [
        { text: 'Annuler', role: 'cancel' },
        {
            text: 'Confirmer',
            handler: (data) => {
                if (!data.email || !data.password) {
                    this.showAlert('Erreur', 'Email et mot de passe
requis.');
```

```

        headers: { 'Authorization': `Bearer
${this.authService.getToken()} ` },

        body: { email: data.email, password: data.password }
    }).subscribe({

        next: () => {

            this.authService.logout();

            this.showAlert('Succès', 'Compte supprimé avec
succès.', () => {

                this.router.navigate(['/login']);

            });

        },

        error: (err) => {

            console.error('Erreur lors de la suppression:',
err);

            const message = err.error?.message || 'Erreur lors
de la suppression. Vérifiez vos identifiants.';

            this.showAlert('Erreur', message);

        }

    });

    return true;

}

}

]

});

await credAlert.present();

```

```

    }

    }

  ]

});

  await confirmAlert.present();

}

```

Résumé des défis rencontrés et solutions

1. Défi : Erreur "TypeError: Cannot read properties of undefined (reading 'toString')"

- **Problème** : Dans home.page.ts, toString() était appelé sur des valeurs undefined lors de la normalisation des horaires, car certains créneaux retournés par /medecins avaient des debut ou fin mal formés (ex. vides).
- **Solution** :
 - Ajout d'une validation regex dans home.page.ts pour filtrer les créneaux valides.
 - Modification de /medecins dans app.py pour ne retourner que les créneaux au format HH:mm avec est_disponible: true.
 - Nettoyage de la collection disponibilite dans MongoDB.

dans home.page.ts:

```

const creneauxValides = creneaux.filter(

  (creneau: any) => creneau.debut && creneau.fin &&
/^\\d{1,2}:\\d{2}$/.test(creneau.debut) &&
/^\\d{1,2}:\\d{2}$/.test(creneau.fin)

);

if (creneauxValides.length === 0) {

```

```

const errorAlert = await this.alertCtrl.create({
  header: 'Erreur',
  message: `Aucun créneau valide disponible pour ${selectedJour}.`,
  buttons: ['OK']
});

await errorAlert.present();

return false;
}

```

dans app.py:

```

filtered_creneaux = [
    {"debut": creneau["debut"], "fin": creneau["fin"]}
    for creneau in creneaux
    if (creneau.get("est_disponible", False) and
        creneau.get("debut") and creneau.get("fin") and
        isinstance(creneau["debut"], str) and isinstance(creneau["fin"],
str) and
        re.match(r"^\d{2}:\d{2}$", creneau["debut"]) and
        re.match(r"^\d{2}:\d{2}$", creneau["fin"]))
]

```

2. Défi : Erreur "Créneau invalide sélectionné"

- **Problème** : Lors de la sélection d'un créneau (ex. "08:00:08:30"), `selectedCreneau.split(':')` produisait 4 éléments (["08", "00", "08", "30"]) au lieu de 2 (["08:00", "08:30"]), déclenchant l'erreur.
- **Solution** : Changement du séparateur de : à | (ex. "08:00|08:30") dans `home.page.ts` pour éviter la confusion avec les deux-points des horaires. Ajustement de la validation avec `split('|')`.

dans `home.page.ts`:

```
const value = `${creneau.debut}|${creneau.fin}`; // Utiliser '|' comme
séparateur

console.log(`Créneau affiché: ${creneau.debut} - ${creneau.fin}, value:
${value}`);

return {

  name: 'creneau',

  type: 'radio',

  label: `${creneau.debut} - ${creneau.fin}`,

  value: value

};

// Validation

const times = selectedCreneau.split('|'); // Diviser sur '|'

if (times.length !== 2) {

  console.error('Invalid times split:', times);

  const errorAlert = await this.alertCtrl.create({

    header: 'Erreur',

    message: 'Créneau invalide sélectionné.',

    buttons: ['OK']
```

```

});

await showAlert.present();

return false;
}

```

3. Défi : Incohérence entre /medecins et /medecins/search

- **Problème** : La route /medecins/search ne validait pas les formats HH:mm ni est_disponible, contrairement à /medecins, causant des données incohérentes.
- **Solution** : Harmonisation de la logique de filtrage dans /medecins/search pour inclure les mêmes validations (regex, est_disponible: true). Ajout de logs dans home.page.ts pour inspecter les données.

dans [app.py](#):

```

filtered_horaires = {}

for jour, creneaux in horaires.items():

    filtered_creneaux = [

        {"debut": creneau["debut"], "fin": creneau["fin"]}

        for creneau in creneaux

        if (creneau.get("est_disponible", False) and

            creneau.get("debut") and creneau.get("fin") and

            isinstance(creneau["debut"], str) and

isinstance(creneau["fin"], str) and

            re.match(r"^\d{2}:\d{2}$", creneau["debut"]) and

            re.match(r"^\d{2}:\d{2}$", creneau["fin"]))

    ]

    if filtered_creneaux:

        filtered_horaires[jour] = filtered_creneaux

```


4. Défi : Créneaux invalides dans MongoDB

Problème : La collection disponibilite contenait des créneaux avec des formats incorrects (ex. "9:0", vides) en raison d'une validation insuffisante dans /disponibilites.

Solution : Ajout d'une validation regex dans /disponibilites pour normaliser les horaires à HH:mm. Exécution d'une requête MongoDB pour nettoyer les créneaux invalides.

```
for creneau in creneaux:

    if ("debut" in creneau and "fin" in creneau and
        isinstance(creneau["debut"], str) and isinstance(creneau["fin"],
str) and
        re.match(r"^\d{1,2}:\d{2}$", creneau["debut"]) and
        re.match(r"^\d{1,2}:\d{2}$", creneau["fin"])):

        hours_debut, minutes_debut = map(int, creneau["debut"].split(':'))
        hours_fin, minutes_fin = map(int, creneau["fin"].split(':'))
        normalized_debut = f"{hours_debut:02d}:{minutes_debut:02d}"
        normalized_fin = f"{hours_fin:02d}:{minutes_fin:02d}"

        updated_creneaux.append({

            "debut": normalized_debut,

            "fin": normalized_fin,

            "est_disponible": True

        })

    else:

        print(f"Créneau invalide ignoré pour {jour}: {creneau}")

////////
```

Requête MongoDB:

```

db.disponibilite.updateMany(
  {},
  [
    {
      $set: {
        horaires: {
          $arrayToObject: {
            $map: {
              input: { $objectToArray: "$horaires" },
              as: "jour",
              in: {
                k: "$$jour.k",
                v: {
                  $filter: {
                    input: "$$jour.v",
                    as: "creneau",
                    cond: {
                      $and: [
                        { $ne: ["$$creneau.debut", null] },
                        { $ne: ["$$creneau.fin", null] },
                        { $regexMatch: { input: "$$creneau.debut", regex:
"^\d{1,2}:\d{2}$" } },
                        { $regexMatch: { input: "$$creneau.fin", regex:
"^\d{1,2}:\d{2}$" } },

```

